



大模型推理加速——KV Cache篇

来自：AiGC面试宝典

Just do it!

2024年06月23日 11:44

大模型推理加速——KV Cache篇

- 一、介绍一下 KV Cache是啥?
- 二、为什么要进行 KV Cache?
 - 2.1 不使用 KV Cache 场景
 - 2.2 使用 KV Cache 场景
- 三、说一下 KV Cache 在 大模型中的应用?
 - 3.1 KV Cache 在 Llama 推理流程中应用?
- 四、KV Cache 优点?
- 五、KV Cache 缺点?
- 六、KV Cache 优化策略?
 - 6.1 PageAttention显存优化
 - 6.2 MHA、GQA、MQA优化技术
 - 6.3 FlashAttention优化技术
- 七、GPT模型单次inference输入生成下一个token，为什么会产生kv-cache?
- 八、为什么kv cache会造成显存很大?

一、介绍一下 KV Cache 是啥?

kv cache 中的 k 和 v 指的分别是 attention 机制中的 key 和 value 的状态值，kv cache 只出现在 transformer 结构的自回归的 decoder 中，像 bert 就没有 kv cache。**kv cache** 的存在是为了避免 **scaled dot-product attention** 过程中的重复计算。

该技术可以在不影响任何计算精度的前提下，通过空间换时间思想，提高推理性能。

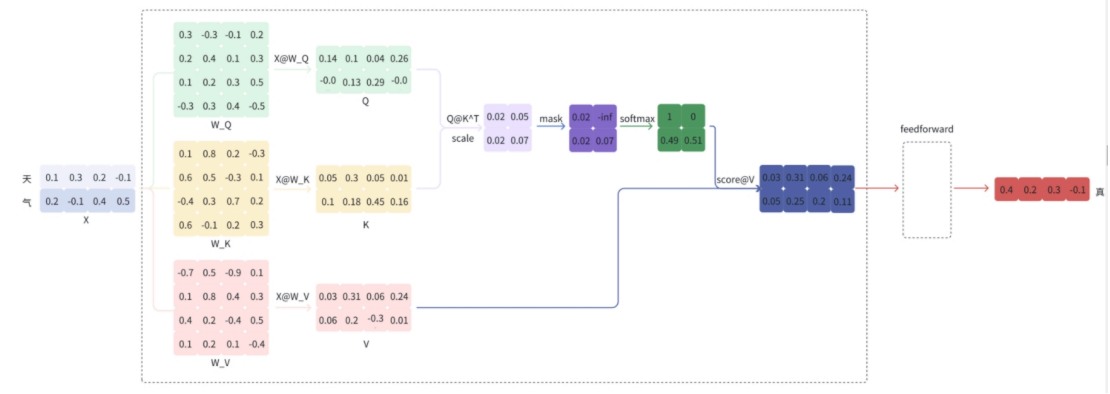
二、为什么要进行 KV Cache?

2.1 不使用 KV Cache 场景

给定“天气”，模型会逐个预测剩下的字，假设接下来预测的两个字为“真好”。

注意：下面的示例图只给出了和 KV Cache 相关的细节。

1. 第一步会预测“真”



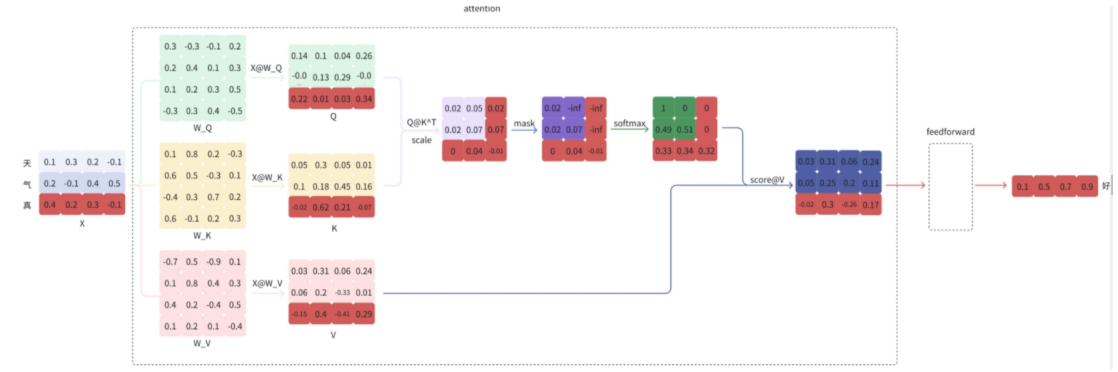
下面是上图计算流程的代码实现：

```
import torch
import torch.nn.functional as F

X = torch.tensor(
    [[0.1, 0.3, 0.2, -0.1],
     [0.2, -0.1, 0.4, 0.5]]
)
W_Q = torch.tensor(
    [[0.3, -0.3, -0.1, 0.2],
     [0.2, 0.4, 0.1, 0.3],
     [0.1, 0.2, 0.3, 0.5],
     [-0.3, 0.3, 0.4, -0.5]]
)
W_K = torch.tensor(
    [[0.1, 0.8, 0.2, -0.3],
     [0.6, 0.5, -0.3, 0.1],
     [-0.4, 0.3, 0.7, 0.2],
     [0.6, -0.1, 0.2, 0.3]]
)
W_V = torch.tensor(
    [[-0.7, 0.5, -0.9, 0.1],
     [0.1, 0.8, 0.4, 0.3],
     [0.4, 0.2, -0.4, 0.5],
     [0.1, 0.2, 0.1, -0.4]]
)
Q = torch.matmul(X, W_Q)
K = torch.matmul(X, W_K)
V = torch.matmul(X, W_V)
scores = torch.matmul(Q, K.T) / 2
scores += torch.tensor(
    [[0, float('-inf')],
     [0, 0]]
)
scores = F.softmax(scores, dim=-1)
output = torch.matmul(scores, V)
```

output 再经过 feedforward 等步骤最终得到预测的 token “真”；

2. 第二步会将“真”拼接“到”天气“的后面，即新的输入为“天气真”，再预测“好”



下面是上图计算流程的代码实现：

```
X = torch.tensor([
    [0.1, 0.3, 0.2, -0.1],
    [0.2, -0.1, 0.4, 0.5],
    [0.4, 0.2, 0.3, -0.1]]
)
Q = torch.matmul(X, W_Q)
K = torch.matmul(X, W_K)
V = torch.matmul(X, W_V)
scores = torch.matmul(Q, K.T) / 2
scores += torch.tensor([
    [0, float('-inf'), float('-inf')],
    [0, 0, float('-inf')],
    [0, 0, 0]
])
scores = F.softmax(scores, dim=-1)
output = torch.matmul(scores, V)
```

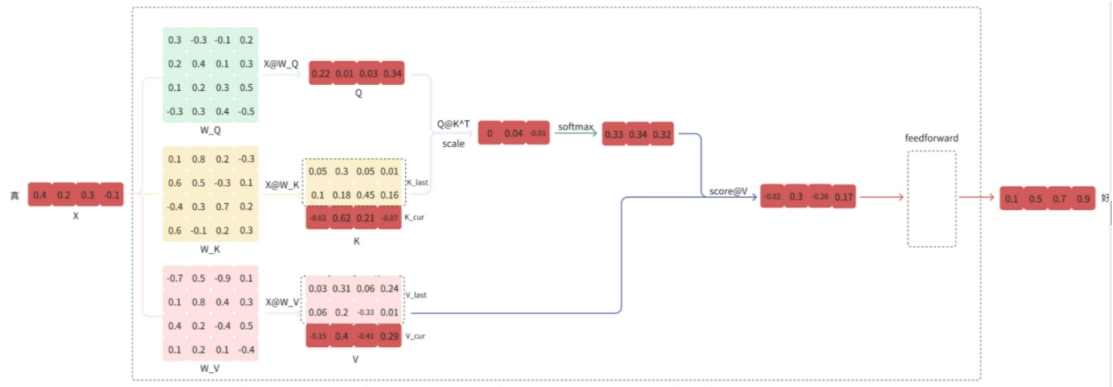
同样的，output 再经过 feedforward 等步骤最终得到预测的 token “好”；

2.2 使用 KV Cache 场景

观察上面的计算过程，可以看到，在第二步的预测中，”好“的预测只和”真“以及完整的 K, V 有关。

于是，KV Cache 的想法就很直观了，缓存上一轮的 **K, V**，即可达到减少计算，提速的效果。从第二步开始时，只需输入当前位置的 token，得到当前位置对应的 K_{cur} , V_{cur} ，再拼接上一步缓存的 K_{last} , V_{last} 得到完整的 K, V ，即可完成下一个 token 的

预测。下图是在上图的基础上只保留和预测”好“相关的数据：



即当前轮输出 **token** 与输入 **tokens** 拼接，并作为下一轮的输入 **tokens**，反复多次。可以看出第 $i+1$ 轮输入数据只比第 i 轮输入数据新增了一个 **token**，其他全部相同！

因此第 $i+1$ 轮推理时必然包含了第 i 轮的部分计算。KV Cache 的出发点就在这里，缓存当前轮可重复利用的计算结果，下一轮计算时直接读取缓存结果，就是这么简单，不存在什么 Cache miss 问题。

三、说一下 KV Cache 在大模型中的应用？

3.1 KV Cache 在 LLaMA 推理流程中应用？

代码来自 <https://github.com/facebookresearch/llama>

LLaMA 类是对模型和 tokenizer 的封装，只实现了 generate 方法，这个方法主要接受 prompt 列表。

```
class LLaMA:
    def __init__(self, model: Transformer, tokenizer: Tokenizer):
        self.model = model
        self.tokenizer = tokenizer

    def generate(
        self,
        prompts: List[str],
        max_gen_len: int,
        temperature: float = 0.8,
        top_p: float = 0.95,
    ) -> List[str]:
        pass
```

下面是一个调用 LLaMA generate 方法的示例：

```
prompts = [
```

```

    "天气",
    "你好",
]

generator = LLaMA(model, tokenizer)
results = generator.generate(prompts)

```

对应的 generate 方法的具体实现：

```

class LLaMA:
    def generate(
        self,
        prompts: List[str],
        max_gen_len: int,
        temperature: float = 0.8,
        top_p: float = 0.95,
    ) -> List[str]:
        bsz = len(prompts)
        params = self.model.params
        assert bsz <= params.max_batch_size, (bsz, params.max_batch_size)

        # step 1: 将 prompt 处理 (tokenizer.encode) 成 prompt_tokens
        prompt_tokens = [self.tokenizer.encode(x, bos=True, eos=False) for x in prompts]
        min_prompt_size = min([len(t) for t in prompt_tokens])
        max_prompt_size = max([len(t) for t in prompt_tokens])
        total_len = min(params.max_seq_len, max_gen_len + max_prompt_size)

        # step 2: 构造一个大小为 (bsz, total_len) 且初始值为 tokenizer.pad_id 的张量 tokens
        tokens = torch.full((bsz, total_len), self.tokenizer.pad_id).cuda().long()

        # step 3: 将 prompt_tokens 赋值给对应位置的 tokens
        for k, t in enumerate(prompt_tokens):
            tokens[k, : len(t)] = torch.tensor(t).long()

        # step 4: 循环预测下一个 token: 只有第一次预测时会将前面所有的 token (例如 "天气 ") 输入给模型, 从第二次预测开始只将当前的 token (例如 "真 ") 输入给模型
        input_text_mask = tokens != self.tokenizer.pad_id
        start_pos = min_prompt_size
        prev_pos = 0
        for cur_pos in range(start_pos, total_len):
            # 调用 模型 forward 函数
            logits = self.model.forward(tokens[:, prev_pos:cur_pos], prev_pos)
            if temperature > 0:
                probs = torch.softmax(logits / temperature, dim=-1)
                next_token = sample_top_p(probs, top_p)
            else:
                next_token = torch.argmax(logits, dim=-1)
            next_token = next_token.reshape(-1)
            # only replace token if prompt has already been generated
            next_token = torch.where(
                input_text_mask[:, cur_pos], tokens[:, cur_pos], next_token
            )
            tokens[:, cur_pos] = next_token
            prev_pos = cur_pos

        # step 5: 预测结束后, 将 token 转成 (tokenizer.decode) 字符串
        decoded = []
        for i, t in enumerate(tokens.tolist()):

```

```

        # cut to max gen len
        t = t[: len(prompt_tokens[i]) + max_gen_len]
        # cut to eos tok if any
        try:
            t = t[: t.index(self.tokenizer.eos_id)]
        except ValueError:
            pass
        decoded.append(self.tokenizer.decode(t))
    return decoded

```

下面介绍 模型的 forward 函数：

```

class Transformer(nn.Module):

    @torch.inference_mode()
    def forward(self, tokens: torch.Tensor, start_pos: int):
        _bsz, seqlen = tokens.shape
        h = self.tok_embeddings(tokens)
        self.freqs_cis = self.freqs_cis.to(h.device)
        freqs_cis = self.freqs_cis[start_pos : start_pos + seqlen]

        mask = None

        # step 1: 对 seqlen 进行判断，只有第一次预测下一个 token 时才会初始化 mask（输入的长度大于
        # 1），因为第二次开始每次只会输入当前位置的 token
        if seqlen > 1:
            mask = torch.full((1, 1, seqlen, seqlen), float("-inf"), device=tokens.device)
            mask = torch.triu(mask, diagonal=start_pos + 1).type_as(h)

        for layer in self.layers:
            h = layer(h, start_pos, freqs_cis, mask)
        h = self.norm(h)
        output = self.output(h[:, -1, :]) # only compute last logits
    return output.float()

```

之所以可以只输入当前位置的 token 就可以预测下一个 token，是因为缓存了 K, V。

具体实现可以看一下 Attention 类：

```

class Attention(nn.Module):
    def __init__(self, args: ModelArgs):
        # 此处省略了和 KV cache 无关的初始化代码
        self.cache_k = torch.zeros(
            (args.max_batch_size, args.max_seq_len, self.n_local_heads, self.head_dim)
        ).cuda()
        self.cache_v = torch.zeros(
            (args.max_batch_size, args.max_seq_len, self.n_local_heads, self.head_dim)
        ).cuda()

    def forward(self, x: torch.Tensor, start_pos: int, freqs_cis: torch.Tensor, mask:
Optional[torch.Tensor]):
        bsz, seqlen, _ = x.shape
        # step 1: 计算当前位置 token 对应的 xq, xk, xv
        xq, xk, xv = self.wq(x), self.wk(x), self.wv(x)

        xq = xq.view(bsz, seqlen, self.n_local_heads, self.head_dim)
        xk = xk.view(bsz, seqlen, self.n_local_heads, self.head_dim)
        xv = xv.view(bsz, seqlen, self.n_local_heads, self.head_dim)

        xq, xk = apply_rotary_emb(xq, xk, freqs_cis=freqs_cis)

```

```
# step 2: 将 xk, xv 缓存到对应的 cache_k, cache_v 中
self.cache_k = self.cache_k.to(xq)
self.cache_v = self.cache_v.to(xq)

self.cache_k[:bsz, start_pos : start_pos + seqlen] = xk
self.cache_v[:bsz, start_pos : start_pos + seqlen] = xv

keys = self.cache_k[:bsz, : start_pos + seqlen]
values = self.cache_v[:bsz, : start_pos + seqlen]

# step 3: 使用 xk 与前面所有的 k 计算 score, 再与 v 进行计算
xq = xq.transpose(1, 2)
keys = keys.transpose(1, 2)
values = values.transpose(1, 2)
scores = torch.matmul(xq, keys.transpose(2, 3)) / math.sqrt(self.head_dim)
if mask is not None:
    scores = scores + mask # (bs, n_local_heads, slen, cache_len + slen)
scores = F.softmax(scores.float(), dim=-1).type_as(xq)
output = torch.matmul(scores, values) # (bs, n_local_heads, slen, head_dim)
output = output.transpose(
    1, 2
).contiguous().view(bsz, seqlen, -1)

return self.wo(output)
```

四、KV Cache 优点?

避免每次采样 token 时重新计算键值向量, 利用预先计算好的 k 值和 v 值, 可以节省大量计算时间

五、KV Cache 缺点?

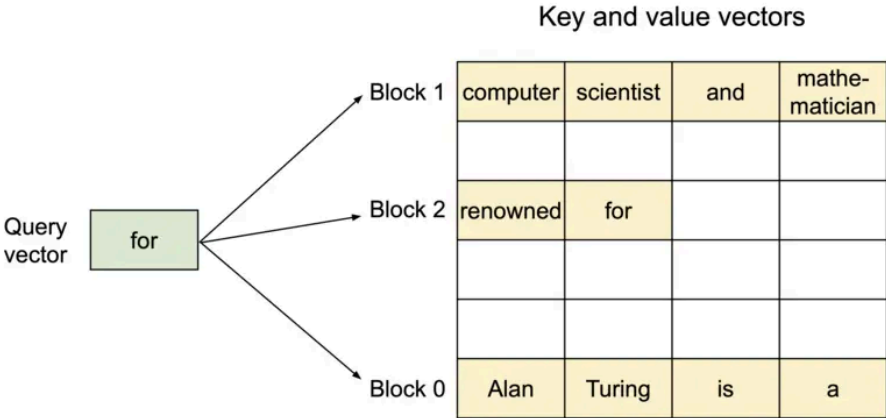
占用一定的存储空间

六、KV Cache 优化策略?

尽可能的减少推理过程中 kv 键值对的重复计算, 实现 kv cache 的优化。目前减少 KV cache 的手段有许多, 比如 page attention、MQA、MGA 等, 另外 flash attention 可以通过硬件内存使用的优化, 提升推理性能。

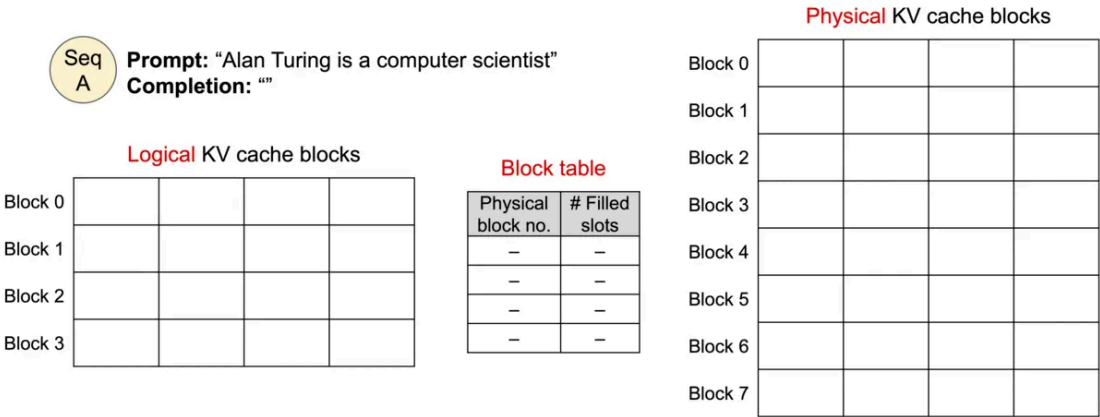
6.1 PageAttention 显存优化

- 动机：在缓存中，这些 KV cache 都很大，并且大小是动态变化的，难以预测。已有的系统中，由于显存碎片和过度预留，浪费了 60%-80% 的显存。
- 解决方法：作为 VLLM 核心技术，PageAttention 通过对 显存碎片化问题进行处理，以达到减少显存占用，提高 KV cache 可使用的显存空间，提升推理性能。
- 优化策略：
 1. PageAttention 借助 OS 系统中虚拟内存和分页的思想。可以实现在不连续的空间存储连续的 kv 键值。



2. 因为所有键值都是分布存储的，需要通过分页管理彼此的关系。序列的连续逻辑块通过 block table 映射到非连续物理块。

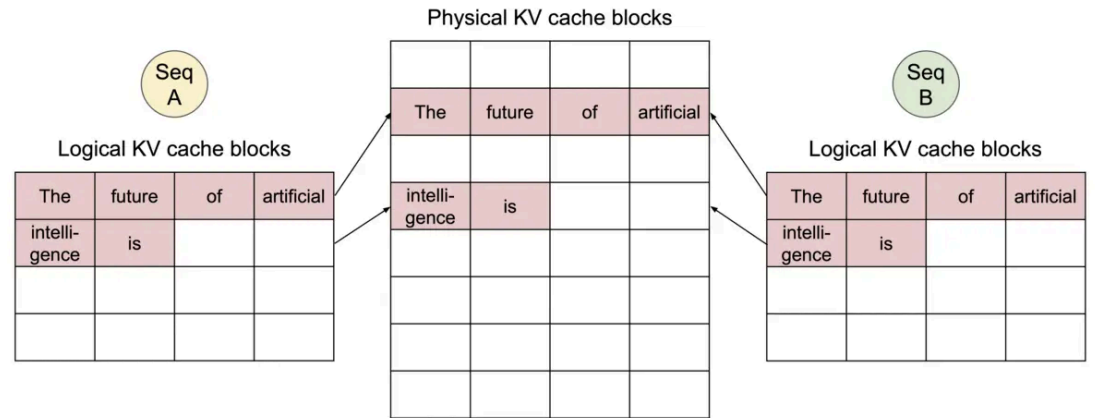
0. Before generation.



3. 同一个 prompt 生成多个输出序列，可以共享计算过程中的 attention 键值，实现 copy-on-write

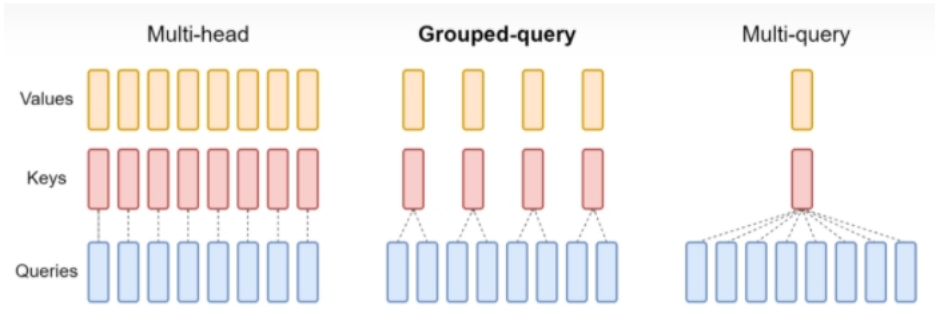
机制，即只有需要修改的时候才会复制，从而大大降低显存占用。

0. Shared prompt: Map logical blocks to the same physical blocks.



6.2 MHA、GQA、MQA 优化技术

- MHA（Multi-head Attention）是标准的多头注意力机制，h 个 Query、Key 和 Value 矩阵。
- MQA 让所有的头之间共享同一份 Key 和 Value 矩阵，每个头只单独保留了一份 Query 参数，从而大大减少 Key 和 Value 矩阵的参数量。
- GQA 将查询头分成 N 组，每个组共享一个 Key 和 Value 矩阵



GQA 以及 MQA 都可以实现一定程度的 Key value 的共享，从而可以使模型体积减小，GQA 是 MQA 和 MHA 的折中方案。

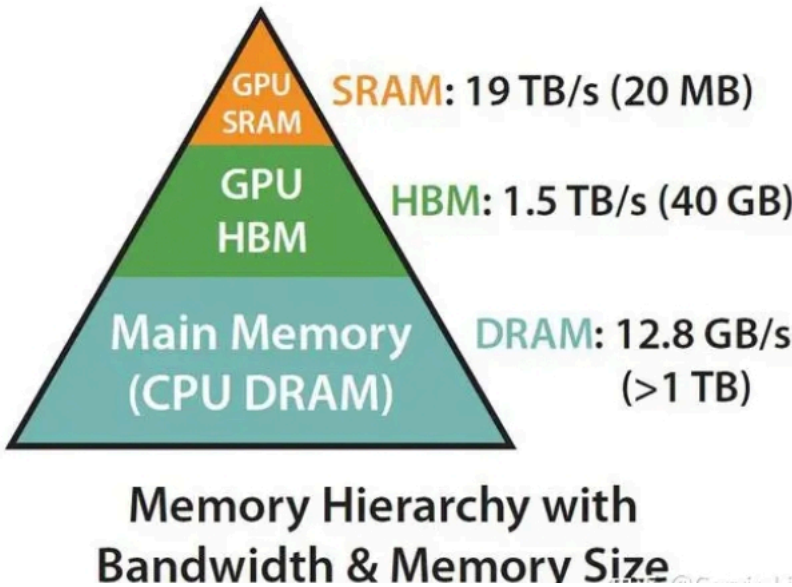
这两种技术的加速原理是

- （1）减少了数据的读取
- （2）减少了推理过程中的 KV Cache。

需要注意的是 GQA 和 MQA 需要在模型训练的时候开启，按照相应的模式生成模型。

6.3 FlashAttention 优化技术

Flash attention 推理加速技术是利用 GPU 硬件非均匀的存储器层次结构实现内存节省和推理加速，意思是通过合理的应用 GPU 显存实现 IO 的优化，从而提升资源利用率，提高性能。



计算速度越快的硬件往往越昂贵且体积越小，Flash attention 的核心原理是尽可能地合理应用 SRAM 内存计算资源。

A100 GPU 有 40-80GB 的高带宽内存(HBM)，带宽为 1.5-2.0 TB/s，而每 108 个流处理器有 192KB 的 SRAM，带宽估计在 19TB/s 左右。也就是说，存在一种优化方案是利用 SRAM 远快于 HBM 的性能优势，将密集计算尽放在 SRAM，减少与 HBM 的反复通信，实现整体的 IO 效率最大化。比如可以将矩阵计算过程，softmax 函数尽可能在 SRAM 中处理并保留中间结果，全部计算完成后再写回 HBM，这样就可以减少 HBM 的写入写出频次，从而提升整体的计算性能。如何有

效分割矩阵的计算过程，涉及到 flash attention 的核心计算逻辑 Tiling 算法。

七、GPT 模型单次 inference 输入生成下一个 token，为什么会产生 kv-cache？

因为 GPT 每次 inference 只能生成一个 token，所以要输出一句话，会进行多次 inference，知道遇到终止 token。这样每次计算输出的 token 的概率分布时，都需要把之前生成的 token 重新计算 key 和 value。第 $i + 1$ 次输入的 token 只比第 i 次输入 token 多一个新的 token，因此第 $i + 1$ 轮推理就包含了第 i 次推理的很多计算。对于第 i 次推理，只需要再计算对应的

k_{i+1} 和 v_{i+1} 即可，所以可以把之前的 k 和 v 缓存下来，不需要每次都计算之前的 k 和 v ，所以就有了 kv cache。这是一种空间换时间的方法。

八、为什么 kv cache 会造成显存很大？

因为随着推理输出的序列变长，需要缓存的 kv cache 也越多。以 GPT3 为例，GPT3 模型占用显存大小为 350GB。假设批次大小 $b=64$ ，输入序列长度 $s=512$ ，输出序列长度 $n=32$ ，则 KV cache 占用显存为 164GB，大约是模型参数显存的 0.5 倍。

